



Objetivo

O objetivo deste trabalho é aplicar os conceitos de estruturas de dados lineares e não lineares estudados em aula, desenvolvendo uma aplicação capaz de implementar e utilizar estruturas do tipo pilha e fila, bem como utilizar algoritmos de ordenação.

Especificações

A implementação deve **simular um posto de atendimento bancário** possuindo as seguintes funcionalidades:

- O programa deve gerenciar duas **filas** distintas de clientes: `FilaPrioridade` e `FilaNormal`.
 - Gerenciamento das filas de clientes implicam a criação de uma classe `RegistroAtendimento` que possui no mínimo identificador do cliente, horário de entrada na fila, tempo de atendimento (valor entre 2 e 30 min para efeitos da simulação) e horário do início de atendimento.
- O programa deve gerenciar três guichês de atendimento: um guichê preferencial (`AtendimentoPreferencial`) e dois guichês gerais (`AtendimentoGeral`).
 - Guichê Preferencial: Deve atender **exclusivamente** clientes da `FilaPrioridade`. Mesmo em caso que esta fila esteja vazia, o guichê não pode atender cliente da `FilaNormal`.
 - Guichês Gerais: Devem seguir uma lógica de alternância da fila normal com a de prioridade. Se houver cliente na `FilaPrioridade`, o guichê prioriza este. Caso tenha acabado de atender um cliente prioritário, deve-se verificar a `FilaNormal`. A lógica de atendimento deve garantir que, após o atendimento de um cliente prioritário em um guichê geral, priorize-se o próximo da fila normal, equilibrando o tempo de espera.
 - Armazenamento de Histórico: Cada guichê deve possuir uma **Pilha** que armazena os objetos dos clientes atendidos por aquele guichê específico, permitindo a consulta dos atendimentos realizados por cada caixa.
- O programa deve ser capaz de:
 - Adicionar Cliente: Inserir um cliente na fila correspondente (Prioritário ou Normal), registrando o id, horário de chegada e tipo.
 - Chamar Próximo: Executar a lógica de prioridade entre os guichês descrita acima, calculando o tempo_espera (diferença entre horário de atendimento e horário de chegada).
 - Imprimir Relatório de Atendimentos: Deve exibir um relatório consolidado contendo:
 - Total de atendimentos realizados e por guichê.
 - Quantidade de atendimentos (Normal vs. Prioritário) por guichê.
 - Métricas de desempenho: Tempo médio de espera total, tempo médio por prioridade e tempo médio por atendimento normal.
 - Relação de Atendimentos: Listar todos os clientes atendidos ordenados de duas formas: Ordem Crescente de Tempo de Espera; e Ordem Cronológica (conforme o horário de atendimento). [Requer a implementação de um **algoritmo de ordenação** (QuickSort ou MergeSort) sobre a base de dados]

Restrições

- Não é permitido utilizar classes de estruturas de dados nativas do Java (p.ex ArrayList, etc.).
- Devem ser reutilizadas as estruturas e algoritmos que implementamos na disciplina.
- É permitido adicionar métodos e atributos complementares.
- Pode ser feito individual ou em equipe de até 2 pessoas.

Entrega

- A entrega deve conter:
 - Diagrama UML da solução
 - Código fonte completo
- Postar solução no AVA até 18h do dia **15/06/2026**

Avaliação

Correção será feita no dia 15/06/2026, com a equipe demonstrando o funcionamento e eventual entrevista com aluno.